Scoring Combine-agent answers: RAGAS vs rubric vs key points

Chirayu Gupta

Vrije Universiteit Brussel

October 2026

RAGAS factual correctness disagrees with the rubric

`gmn-control-region-1`, mean of 3 runs per configuration

agent	FC recall	rubric (1–5)
GPT-5.6 luna, 3 configs	0.43–0.55	5.0
GPT-5.6 terra, 3 configs	0.41–0.66	5.0
Qwen 3.8, read-only config	0.55	3.7

- 37 answers got rubric 5/5; their FC ranges 0.22 to 0.92
- reading them: the GPT answers are correct

What `FactualCorrectness(mode="recall")` computes
(ragas 0.2.15, `_factual_correctness.py`)

$$\text{recall} = \frac{TP}{TP + FN}$$

- TP: claims split from the **answer** that the gold supports
- FN: claims split from the **gold** missing in the answer
- two different LLM splits, redone on every call
- every gold claim weighs the same, side remarks included
  (e.g. the docs' "∼50%" comment in gmn)

Not "fraction of gold covered": it moves with answer length and claim split

Replacement: per-question key points (“nuggets”)

Written once per question, frozen, versioned (`nuggets.yaml`):

`gmn-control-region-1`	
vital	use the `gmN` uncertainty type
vital	CR count N written after the keyword
vital	extrapolation factor α in the process column
vital	rate $= N\alpha$
okay	concrete datacard line; $1/\sqrt{N+1} \approx 45\%$
must_not	recommends `lnN`/`lnU`; example card inconsistent

Judge (Qwen 3.8 on aigw) per item:

- supported / partly / not supported
- must quote the answer; harness checks the quote is really there
- gold prose not shown; items shuffled; 3 judge runs, majority

Scores per answer:

$$V_{\text{strict}} = \frac{\#\,\text{vital fully supported}}{\#\,\text{vital}}$$

pass = all vital supported **and** no must_not present

Method of the TREC 2024/25 RAG track (AutoNuggetizer), plus evidence quotes and must_not items

Results: gmN question (8 agent configurations × 3 runs)

agent	config	nugget pass	V_{strict}	FC recall	rubric (overall)
Qwen 3.8	shipped	0/3	0.00	0.00	1.0
	read-only	2/3	0.67	0.55	3.7
GPT-5.6 luna	shipped	3/3	1.00	0.55	5.0
	read-only	3/3	1.00	0.45	5.0
	no web	3/3	1.00	0.43	5.0
GPT-5.6 terra	shipped	2/3	1.00	0.66	5.0
	read-only	3/3	1.00	0.41	5.0
	no web	3/3 *	1.00	0.59	5.0

shipped = 0.5.4 (bash `ask`); read-only = bash `deny` + read-only allow-list; no web = read-only, web fetch/search denied. Killed/empty runs get the lowest score (0, or 1 for the rubric). Rubric: all four criteria got the same score in 38/41 answers, so only overall is shown.

- correct GPT answers: $V_{\text{strict}} = 1.00$ and rubric 5, while FC gives 0.41–0.66
- terra shipped 2/3: one example card has no *N* after `gmN` (mn3); * a wrong answer the judge passed (rate 10.0, but $25 \times 0.12 = 3.0$)

Results: shape + counting question (8 agent configurations × 3 runs)

agent	config	nugget pass	V_{strict}	FC recall	rubric (overall)
Qwen 3.8	shipped	0/3	0.00	0.00	1.0
	read-only	1/3	0.33	0.44	2.7
GPT-5.6 luna	shipped	2/3	0.67	0.48	3.7
	read-only	1/3*	0.50	0.60	3.7
	no web	3/3	1.00	0.84	5.0
GPT-5.6 terra	shipped	3/3	1.00	0.85	5.0
	read-only	3/3	1.00	0.80	5.0
	no web	2/3*	0.83	0.84	5.0

shipped = 0.5.4 (bash `ask`); read-only = bash `deny` + read-only allow-list; no web = read-only, web fetch/search denied. Killed/empty runs get the lowest score (0, or 1 for the rubric). Rubric: all four criteria got the same score in 38/41 answers, so only overall is shown.

- luna shipped, read-only: one run each killed by a permission rejection (lowest score in all three methods)
- * a correct answer the judge failed (kp2 missed). Overall: nugget pass agrees with rubric ≥ 4 on 45/48 answers, FC ≥ 0.5 on 38/48

What the nuggets add: a named reason for every failure

Caught, rubric said 5/5

- terra shipped config, gmn run 3: example card `bkg_stat gmN - 0.12` (no *N* after gmN) → must_not "inconsistent example" present

Diagnosed

- killed luna runs (permission reject, path typo): miss both vital points, not "low recall"
- Qwen read-only, shape run 2: "shape and counting cards cannot be combined" → must_not

Missed / wrong (judge limits)

- terra no-web config, gmn run 3: `rate 10.0` with `gmN 25 0.12` ($25 \times 0.12 = 3.0$): arithmetic not caught
- 2 correct shape-counting answers failed: judge padded its quote with text not in the answer, quote check rejected it

rubric-vs-nugget disagreements	3/48
real error found by nuggets	1
judge false negative	2

Caveats and next steps

Caveats

- only 2 questions; key points written after seeing answers
- not yet calibrated against human labels
- Qwen judge grades Qwen answers (self-preference)
- 2 vital points on shape-counting: one judge flip $= \pm 0.5$
- agreement with humans is good for ranking systems, weak per question (TREC: τ 0.73–0.90 vs 0.3–0.66)

Next

- key points for all questions, drafted automatically from the **gold only** (LLM ensemble), then reviewed
- must_not items mined from observed wrong answers
- one-off calibration vs the 13 Oct. hand grades (Scott's π, judge leniency)
- cross-family judge for headline numbers
- report pass@1, pass^3, errors clustered by question

Proposal: nugget pass and V_{strict} as headline, rubric as second opinion, drop FC recall

References (scoring)

- E. Voorhees, *Overview of the TREC 2003 QA track*: vital/okay nuggets
- J. Lin, D. Demner-Fushman, *Will pyramids built of nuggets topple over?*, HLT-NAACL 2006
- R. Pradeep et al., *Initial nugget evaluation results for the TREC 2024 RAG track (AutoNuggetizer)*, arXiv:2411.09607
- R. Pradeep et al., *The Great Nugget Recall*, SIGIR 2025, arXiv:2504.15068
- S. Upadhyay et al., *Overview of the TREC 2025 RAG track*, arXiv:2603.09891
- R. Arora et al., *HealthBench*, arXiv:2505.08775 A. Gunjal et al., *Rubrics as Rewards*, arXiv:2507.17746
- Y. Lee et al., *CheckEval*, EMNLP 2025, arXiv:2403.18771
- M. Wanner et al., *A closer look at claim decomposition*, arXiv:2403.11903
- A. S. Thakur et al., *Judging the judges*, arXiv:2406.12624 A. Panickssery et al., arXiv:2404.13076
- S. Yao et al., τ-*bench* (pass^k), arXiv:2406.12045 E. Miller, *Adding error bars to evals*, arXiv:2411.00640

All answers

Whole answers: how to read the next slides

- For each question: the question and its key points, then all 24 answers to it (the 8 agent configurations of the results table $\times$ 3 runs), word for word as stored by the harness (blank lines dropped, long lines wrapped)
- Title: question, model, agent config, run; line below: RAGAS FC recall, rubric (1–5), nugget verdict
- blue[kp]: supports a key point orange[mn]: wrong statement (must_not)
- grey[kp judge missed], orange[mn judge missed]: the judge got it wrong
- Highlights are hand-placed for the answers discussed in the talk; elsewhere they mark the first text matching each key point the judge accepted

Configs: shipped = combine-assistant 0.5.4 as released (bash `ask`); read-only = bash `deny` plus a read-only allow-list; no web = read-only plus web fetch/search denied. Qwen was run with shipped and read-only only.

gmN: the question and its key points

Question (`gmn-control-region-1`)

One of my backgrounds is estimated from the number of events observed in a control region, multiplied by an extrapolation factor into the signal region. Which uncertainty type should I use in the datacard for its statistical uncertainty, and how do I write it?

kp1	vital	use the `gmN` uncertainty type
kp2	vital	control-region count N written after the keyword
kp3	vital	extrapolation factor α in the process column
kp4	vital	rate $= N\alpha$
kp5	okay	a concrete datacard line
kp6	okay	relative uncertainty $\approx 1/\sqrt{N+1}$
mn1	must_not	recommends `lnN`/`lnU` instead
mn2	must_not	number after `gmN` is α or the yield
mn3	must_not	example card inconsistent (rate $\neq N\alpha$, or no N)

Key points written once from the gold answer; the judge sees only these lines. Next: 24 answers to this question, in full.

gmN: Qwen 3.8, shipped config, run 1/3

FC – rubric – nugget fail (no key point supported) 7 tool calls

```
(no answer: the session ended without a final message; 7 tool calls)
```

gmN: Qwen 3.8, shipped config, run 2/3

FC – rubric – nugget fail (no key point supported) 9 tool calls

```
(no answer: the session ended without a final message; 9 tool calls)
```

gmN: Qwen 3.8, shipped config, run 3/3

FC – rubric – nugget fail (no key point supported) 7 tool calls

```
(no answer: the session ended without a final message; 7 tool calls)
```

gmN: Qwen 3.8, read-only config, run 1/3

FC 0.87 rubric 5 nugget pass (supported: kp1, kp2, kp3, kp4, kp5, kp6) 5 tool calls

Use the **`gmN`** (Gamma) uncertainty type[kp1] -- it is the recommended choice in Combine for the statistical uncertainty of a background estimated from the number of events in a control region (or a limited MC sample).
How to write it (matrix-format datacard):
```
WW_norm gmN 4 - 0.16 - -[kp5]  control region: 4 events, extrapolation factor 0.16
```
The rule, from the docs:
- Put **`N`** (the number of events observed in the control region) **immediately after the `gmN`[kp2] keyword** -- here `4`.
- Put **`$\alpha$`** (the extrapolation factor[kp3] from the control region into the signal region) in the relevant **(bin, process) column** -- here `0.16`.
- The **`rate`** value on the `rate` line for that same (bin, process) must equal **`N·$\alpha$`**[kp4] -- here `4 × 0.16 = 0.64`.
So if your background is `N` control-region events times an extrapolation factor `$\alpha$`, the `rate` line says `N·$\alpha$` and the uncertainty line says `gmN N ... $\alpha$ ...`.
For `N = 4`, the resulting `1$\sigma$` relative uncertainty on the extrapolated yield is `1/$\sqrt{}$(N+1)` = `1/$\sqrt{}$5` ≈ 45%[kp6] (the docs' example comment rounds this to "~50%").
This is the counting-experiment example in the Combine docs: [A simple counting experiment](https://cms-analysis.github.io/HiggsAnalysis-CombinedLimit/latest/part2/settinguptheanalysis/#a-simple-counting-experiment).
One note (not from the fetched section, but worth checking): `gmN` is the right tool when the background is a **single number** derived from a control region. If your background is a **shape** (multi-bin) built from a control region, the modern approach is to link the signal- and control-region channels with a rate parameter / `RooParametricHist` so the control-region data statistically constrains the shape -- see [Rate parameters](https://cms-analysis.github.io/HiggsAnalysis-CombinedLimit/latest/model_building_tutorial2024/model_building_exercise/#rate-parameters). Let me know which case you're in and I can go deeper.

gmN: Qwen 3.8, read-only config, run 2/3, page 1/2

FC 0.78 rubric 5 nugget pass (supported: kp1, kp2, kp3, kp4, kp5, kp6) 7 tool calls

```
I'll research the right uncertainty type for a control-region-derived background in the Combine docs.
The "simple counting experiment" doc looks most relevant. Let me fetch it.
The section extraction cut off early. Let me get the full page.
I have the exact answer from the official docs.
## Use a `gmN` (Gamma) nuisance
For the **statistical** uncertainty of a background estimated from the number of events in a control region (or an MC
sample with limited stats), the recommended modifier is **`gmN`**[kp1] -- a Gamma-distributed nuisance. This is *not* `lnN`
(log-normal, which is for multiplicative systematics like efficiencies/cross sections) and not a `shape`/`param`
Gaussian.
The docs state it directly:
> **`gmN`** stands for Gamma distribution, and is the recommended choice for the statistical uncertainty in a
background determined from the number of events in a control region (or in an MC sample with limited sample size).
### How to write it
If the control region contains **N** events and the extrapolation (transfer) factor from the control region to the
signal region is **α**:
1. Put **N** immediately after the `gmN` keyword.[kp2]
2. Put **α** in the relevant (bin, process) column[kp3] of the uncertainty line.
3. The yield in the `rate` line for that (bin, process) must equal **N·α**[kp4].
```
...
process   SR
----
signal   10
bkg_CR   6.4   <- N*alpha = 40 * 0.16
----
stat uncertainty
bkg_CR_stat gmN 40 0.16   # 40 events in CR, transfer factor 0.16
```
```

gmN: Qwen 3.8, read-only config, run 2/3, page 2/2

FC 0.78 rubric 5 nugget pass (supported: kp1, kp2, kp3, kp4, kp5, kp6) 7 tool calls

````
```
Here N = 40 events observed in the control region and α = 0.16, so the expected SR yield is 40 × 0.16 = 6.4. The
implied 1σ relative uncertainty is 1/√(N+1) ≈ 15.4% (the Gamma constraint on the CR count).
### Worked example from the docs
The canonical counting-experiment card uses exactly this: a `qqWW` background from a sideband with 4 events and
extrapolation factor 0.16:
```
WW_norm  gmN  4  -  0.16  -  -[kp5]
```
> The `qqWW` background comes from a sideband with 4 observed events and an extrapolation factor of 0.16; the
resulting uncertainty in the expected yield is 1/√(4+1) = 45%[kp6].
### Caveat
`gmN` models only the *statistical* uncertainty of the CR count. If the extrapolation factor α itself has an
independent (systematic) uncertainty, model that separately (e.g. an additional `lnN` on α). Also note that for a
fully correlated treatment where the CR is fit simultaneously with the SR, the `rateParam` approach is the more
general option -- but for the simple "multiply CR count by a fixed factor" case you describe, `gmN` is the correct and
recommended choice.
**Source:** [Preparing the datacard -- "A simple counting experiment" / Nuisance modifiers](https://cms-
analysis.github.io/HiggsAnalysis-CombinedLimit/latest/part2/settinguptheanalysis/)
````
````

FC 0.00 rubric 1 nugget fail (no key point supported) 0 tool calls

invented tool output: no tool calls, the answer is a made-up skill file

invented, no tool was called

```
<skill_content name="combine">
# CMS Combine
Answer questions about CMS Combine (HiggsAnalysis-CombinedLimit) and, when
helpful, reproduce or diagnose Combine runs.
Two MCP servers back this skill:
- **`combine-mcp`** -- read-only retrieval over four complementary sources:
  the official docs, the methodology paper (arXiv:2404.06614), the source
  code (pinned to a release), and the cms-talk Q&A forum. Tools:
  `search_docs`, `fetch_doc`.
- **`combine-run-mcp`** -- runs a single Combine-family command
  (`combine`, `combineTool.py`, `text2workspace.py`, `combineCards.py`) in an
  isolated workspace. Tool: `run_combine`.
## Routing -- pick the right source first
- Concrete `combine` CLI question or option lookup → `combine-docs`.
- "Why does it work this way?" / methodology / definitions → `combine-paper`.
- "What does the code actually do here?" → `combine-code`.
- "I'm seeing this error/warning" / "anyone else hit this?" → `combine-forum`
  first; if nothing relevant, then `combine-hypernews` (archived, pre-2022,
  may reference outdated versions).
When the first source comes up short, fall through to the next most likely
one -- the sources are complementary. For forum questions the order is
cms-talk (`combine-forum`) then the archived `combine-hypernews`.
## Retrieval
```

FC 0.00 rubric 1 nugget fail (no key point supported) 0 tool calls

invented tool output: no tool calls, the answer is a made-up skill file

```
`search_docs(query, source, limit=10)` returns titles, URLs, paths, scores,
snippets -- no bodies. Call `fetch_doc(url_or_path, source, mode=...)` for the
body. `mode` is source-dependent:
- All sources: `"markdown"` (default, full body), `"outline"` (headings /
  top-level defs / per-post summary), `"sections:<heading>"` (one named
  section, docs/paper).
- `combine-forum` only: `"post:<N>"` (one post, with a per-post URL),
  `"post:accepted"` (the accepted-answer post; errors if the thread isn't
  marked solved).
Typical flows:
Option lookup:
1. `search_docs("what does --robustFit do", source="combine-docs")`
2. `fetch_doc(<url>, source="combine-docs", mode="sections:Common options")`
Error / warning diagnosis:
1. `search_docs("<error message snippet>", source="combine-forum")`
2. `fetch_doc("<topic id>", source="combine-forum", mode="outline")`
3. `fetch_doc("<topic id>", source="combine-forum", mode="post:accepted")`
Methodology / definitions:
1. `search_docs("<concept>", source="combine-paper")`
2. `fetch_doc("<section id>", source="combine-paper")`
Code drill-down:
1. `search_docs("<class or function name>", source="combine-code")`
2. `fetch_doc("<relpath>", source="combine-code", mode="outline")`
```

gmN: Qwen 3.8, read-only config, run 3/3, page 3/4

FC 0.00 rubric 1 nugget fail (no key point supported) 0 tool calls

invented tool output: no tool calls, the answer is a made-up skill file

```
3. `fetch_doc("<relpath>", source="combine-code", mode="sections:<heading>")`
## Running Combine -- local first, remote as fallback
Run a Combine command in the **shell** when Combine is on the user's `PATH`
(their real files, in place). Otherwise use the **`combine-run-remote`**
server's `run_combine` tool, which runs a single command in an isolated
workspace.
`run_combine(command, files=None, files_b64=None, timeout_s=None)`:
- `command`: the full command line, e.g.
  `"combine -M AsymptoticLimits -d datacard.txt -m 120"`.
- `files`: text inputs as `{filename: content}` (datacards, configs).
- `files_b64`: binary inputs as `{filename: base64}` (ROOT shape files).
- `timeout_s`: optional; clamped to the server's ceiling.
Returns JSON: `returncode`, `stdout`, `stderr` (tails, with truncation
flags), `artifacts` (output files produced, by name + size), `elapsed_s`,
`timed_out`, and `error` (set only for setup failures like a disallowed
executable or a missing Combine install).
## Typical use -- reproduce and diagnose
The main use case is debugging: a user reports a Combine command that
errors or gives a wrong result. Reproduce it here, then cross-check the
output against the `combine-mcp` sources (docs / code / forum) to
explain and fix.
1. `run_combine(command=<their command>, files={<their datacard>})`.
2. Read `returncode` / `stderr`. If it failed, search `combine-mcp`
```

gmN: Qwen 3.8, read-only config, run 3/3, page 4/4

FC 0.00 rubric 1 nugget fail (no key point supported) 0 tool calls

invented tool output: no tool calls, the answer is a made-up skill file

```
   (`source="combine-forum"` for the error text, `source="combine-code"`
   for what the code checks) to explain the cause.
3. Propose a corrected command and, if useful, re-run it to confirm.
## Local vs remote
The same tool may be served two ways:
- **local** (stdio, on the user's machine): no upload limits, longer
  timeouts. Requires a working Combine environment where the server
  process runs.
- **remote** (this server, if reached over HTTP on CERN PaaS): shared
  resource, so inputs are size-capped and timeouts are shorter. For
  large datacards or long fits, prefer a local server.
If `error` says the executable was not found on PATH, the server's
process has no Combine environment sourced -- tell the user rather than
guessing at results.
</skill_content>
```

gmN: GPT-5.6 luna, shipped config, run 1/3

FC 0.71 rubric 5 nugget pass (supported: kp1, kp2, kp3, kp4, kp5, kp6) 14 tool calls

Use a **`gmN`[kp1]** nuisance, which models the Poisson counting uncertainty from the control-region yield. It is preferred for a background estimated from `N` control-region events and an extrapolation factor `alpha` into the signal region.
Write the nuisance line as:

```text
background_CR_stat  gmN  N[kp2]  ... alpha ...
```

where:
- `N` is the observed number of control-region events, placed immediately after `gmN`
- `alpha` is the control-to-signal-region extrapolation factor in the relevant process/channel column[kp3]
- the corresponding nominal `rate` must be `N * alpha`[kp4]
- use `-` in columns unaffected by this nuisance

For example, 25 control-region events with an extrapolation factor of `0.20` gives an expected signal-region yield of `5`:

```text
bin      signal_region
process  signal  data_driven_bkg
rate     2.1     5.0
background_CR_stat  gmN  25  -  0.20[kp5]
```

The Combine documentation gives the analogous example `WW_norm gmN 4 ... 0.16`, where the rate is `4 * 0.16 = 0.64`; it also notes that the resulting relative uncertainty is approximately `1/sqrt(N+1[kp6])`. See [Preparing the datacard](https://cms-analysis.github.io/HiggsAnalysis-CombinedLimit/latest/part2/settinguptheanalysis/#a-simple-counting-experiment).

gmN: GPT-5.6 luna, shipped config, run 2/3

FC 0.43 rubric 5 nugget pass (supported: kp1, kp2, kp3, kp4, kp5) 11 tool calls

````
Use a **`gmN`[kp1]** nuisance. It models the Poisson statistical uncertainty from the finite number of events observed in
the control region.
If the control region contains `N` events and the extrapolation factor to the signal region is `alpha`, write:
```text
name gmN N[kp2]  ...  alpha  ...
```
The corresponding nominal background rate must be:
```text
rate = N * alpha[kp4]
```
For example, for 4 control-region events and an extrapolation factor of `0.16`:
```text
bin      signal_region
process  background
process  1
rate     0.64
cr_stat  gmN 4  0.16[kp5]
```
Here, `cr_stat gmN 4 0.16` gives a nominal yield of `4 * 0.16 = 0.64`; the statistical uncertainty is handled through
the gamma constraint rather than an `lnN` percentage. For unaffected process/channel columns, use `-`.
Source: [Combine documentation, "A simple counting experiment"](https://cms-analysis.github.io/HiggsAnalysis-
CombinedLimit/latest/part2/settinguptheanalysis/#a-simple-counting-experiment)
````

gmN: GPT-5.6 luna, shipped config, run 3/3

FC 0.50 rubric 5 nugget pass (supported: kp1, kp2, kp3, kp4, kp5, kp6) 10 tool calls

````
Use a `gmN` kp1 nuisance, which models the Poisson counting uncertainty from the control-region yield.
If the control region contains `N` observed events and the transfer/extrapolation factor to the signal
region is `alpha`, write:
```text
nuisance_name  gmN  N kp2  ...  alpha  ...
```
The affected process's nominal rate must be:
```text
rate = N * alpha kp4
```
For example, 4 control-region events with an extrapolation factor of `0.16` give:
```text
bin      signal  background
process  0       1
rate     1.0     0.64
cr_stat  gmN  4   -       0.16 kp5
```
Here the background estimate is `4 * 0.16 = 0.64`. The `gmN` uncertainty is preferable to an `lnN`
approximation because it retains the correct counting-statistics model. The Combine documentation notes that
the corresponding approximate relative uncertainty is \(1/\sqrt{N+1 kp6}\), about 45% for \(N=4\).
[Combine documentation: A simple counting experiment](https://cms-analysis.github.io/HiggsAnalysis-
CombinedLimit/latest/part2/settinguptheanalysis/#a-simple-counting-experiment)
````

gmN: GPT-5.6 luna, read-only config, run 1/3

FC 0.60 rubric 5 nugget pass (supported: kp1, kp2, kp3, kp4, kp5) 13 tool calls

````
Use a **`gmN`[kp1]** nuisance, which is intended for statistical uncertainties from a finite event count in a
control region or limited MC sample.
If the control region contains `N` events and the extrapolation factor to the signal region is `alpha`,
write:
```text
<name>  gmN  N[kp2]  -  ...  alpha  ...
```
The affected process's nominal rate must be:
```text
rate = N * alpha[kp4]
```
For example, 25 control-region events with an extrapolation factor of `0.20`:
```text
bin       sr
process   bkg
rate      5.0
bkg_CR_stat  gmN  25  0.20[kp5]
```
Here, `bkg_CR_stat` represents the Poisson/Gamma statistical uncertainty from the 25 control-region events.
Do not use `lnN` for this counting-statistics component; reserve `lnN` for multiplicative uncertainties such
as efficiencies or cross sections.
[Combine datacard documentation: `gmN` uncertainties](https://cms-analysis.github.io/HiggsAnalysis-
CombinedLimit/latest/part2/settinguptheanalysis/#a-simple-counting-experiment)
````

gmN: GPT-5.6 luna, read-only config, run 2/3

FC 0.40 rubric 5 nugget pass (supported: kp1, kp2, kp3, kp4) 10 tool calls

````
Use a **`gmN`[kp1]** nuisance, which models the Poisson statistical uncertainty from the control-region event
count.
If the control region contains `N` events and the extrapolation factor to the signal region is `alpha`,
write:
```text
background_CR_stat  gmN  N[kp2]  -  alpha
```
The corresponding nominal background rate in the `rate` line must be:
```text
N * alpha[kp4]
```
For example, with 4 control-region events and an extrapolation factor of `0.16`:
```text
bin      signal_region
process  signal  background
rate     2.0     0.64
background_CR_stat  gmN  4  -  0.16[kp5]
```
Here, Combine treats the background yield as `4 * 0.16 = 0.64`. The `gmN` choice is specifically recommended
for a background estimated from a control region or limited-size simulated sample. The documentation's
example uses the same structure: [`gmN` control-region example](https://cms-
analysis.github.io/HiggsAnalysis-CombinedLimit/latest/part2/settinguptheanalysis/#a-simple-counting-
experiment).
````

gmN: GPT-5.6 luna, read-only config, run 3/3

FC 0.36 rubric 5 nugget pass (supported: kp1, kp2, kp3, kp4, kp5) 7 tool calls

I'll verify the Combine datacard syntax for control-region yield estimates and distinguish the appropriate statistical uncertainty from rate/systematic uncertainties. Use a **`gmN`** uncertainty**. It models the Poisson statistical uncertainty from the finite number of events observed in the control region.
If the control region contains `N` events and the extrapolation factor to the signal region is `alpha`, write:

```text
background_stat  gmN  N  alpha
```

The corresponding nominal background rate must be:

```text
rate  N * alpha
```

For example, with 4 control-region events and an extrapolation factor of `0.16`:

```text
bin           signal_region
process       sig  background
rate          10   0.64
background_stat  gmN  4  -  0.16
```

Here, `-` means the nuisance does not affect the signal, and `4 * 0.16 = 0.64` is the background prediction.
Do not use `lnN` for this Poisson counting uncertainty. See the Combine documentation on [a simple counting experiment](https://cms-analysis.github.io/HiggsAnalysis-CombinedLimit/latest/part2/settinguptheanalysis/#a-simple-counting-experiment).

gmN: GPT-5.6 luna, no web config, run 1/3

FC 0.57 rubric 5 nugget pass (supported: kp1, kp2, kp3, kp4, kp5) 11 tool calls

I'll check the Combine documentation for the control-region extrapolation case and the exact datacard syntax, then distinguish the statistical treatment from any separate extrapolation-factor uncertainty.Use a **`gmN`**[kp1] nuisance, which represents the Poisson statistical uncertainty from the \(N\) events observed in the control region.

The syntax is:

```text
<name>  gmN  N[kp2]  <alpha for process 1>  <alpha for process 2> ...
```

Here:

- `N` is the number of events observed in the control region.
- `alpha` is the extrapolation factor from the control region to the signal region.
- The nominal predicted yield is \(N \times \alpha[kp4]\).
- Use `-` for processes unaffected by this uncertainty.

For example, if 20 events are observed in the control region and the transfer factor is 0.35:

```text
crStat  gmN  20  0.35  -[kp5]
```

This gives the background a nominal signal-region yield of \(20 \times 0.35 = 7\), with the corresponding Poisson/gamma statistical uncertainty from the control-region count. Do not encode this as an `lnN`; `gmN` preserves the counting-statistics interpretation.

See the Combine documentation's [simple counting experiment](https://cms-analysis.github.io/HiggsAnalysis-CombinedLimit/latest/part2/settinguptheanalysis/#a-simple-counting-experiment), which specifically describes a control-region event count combined with an extrapolation factor.

gmN: GPT-5.6 luna, no web config, run 2/3

FC 0.50 rubric 5 nugget pass (supported: kp1, kp2, kp3, kp4, kp5) 12 tool calls

````
Use a **`gmN`[kp1] nuisance**, which models the Poisson statistical uncertainty from the control-region event
count.
If the control region contains \(N\) events and the extrapolation factor to the signal region is \(\alpha\),
write:
```text
<name>  gmN  N[kp2]  ...  alpha  ...
```
For example, with 4 control-region events and an extrapolation factor of 0.16:
```text
WW_norm  gmN  4  -  0.16  -  -[kp5]
```
The affected background yield is therefore \(4 \times 0.16 = 0.64\), and the uncertainty is derived from the
Poisson constraint on the observed count of 4 events, rather than approximated as an `lnN` uncertainty. Use
`-` for processes or channels not affected by this constraint.
This is the pattern shown in the [Combine counting-experiment documentation](https://cms-
analysis.github.io/HiggsAnalysis-CombinedLimit/latest/part2/settinguptheanalysis/#a-simple-counting-
experiment).
````

gmN: GPT-5.6 luna, no web config, run 3/3

FC 0.22 rubric 5 nugget pass (supported: kp1, kp2, kp3, kp4, kp5) 11 tool calls

````
Use a **`gmN`** nuisance parameter[kp1]. It models the Poisson statistical uncertainty from the finite number of events in
the control region, rather than approximating it with a symmetric `lnN` uncertainty.
If the control region contains \(N\) events and the extrapolation factor to the signal region is \(\alpha\), write:
```text
name        gmN   N      ...
```
The entry for the relevant background process is \(\alpha\):
```text
bin          signal
process      signal  ttbar
process      0       1
rate         10.0    6.0
crStat       gmN     20       -        0.30[kp5]
```
Here:
- `20` is the observed control-region count[kp2].
- `0.30` is the extrapolation factor[kp3].
- The nominal signal-region background yield is therefore \(20 \times 0.30 = 6.0\)[kp4].
- `-` means the nuisance does not affect the signal.
Do **not** enter `1/sqrt(N)` as an `lnN` value for this case. The `gmN` constraint retains the correct Poisson
behavior, especially for small control-region counts. Any separate uncertainty on the extrapolation factor itself
should generally be represented by its own nuisance, often an `lnN`.
See the Combine documentation's [simple counting experiment](https://cms-analysis.github.io/HiggsAnalysis-
CombinedLimit/latest/part2/settinguptheanalysis/#a-simple-counting-experiment), which specifically recommends this
treatment for backgrounds determined from control-region event counts and an extrapolation factor.
````

gmN: GPT-5.6 terra, shipped config, run 1/3

FC 0.70 rubric 5 nugget pass (supported: kp1, kp2, kp3, kp4, kp5) 2 tool calls

Use a `gmN` [kp1] nuisance. It is the appropriate constraint for the Poisson statistical uncertainty from a control-region count propagated through an extrapolation (transfer) factor.
If the control region has \(N\) observed events and the signal-region prediction is \(N\alpha[kp4]\), write:

```text
crStat  gmN  N[kp2]   -   alpha   -
```

Here, `N` immediately follows `gmN`, and `alpha` goes in the column[kp3] for the affected signal-region background process. The corresponding `rate` entry for that process must be `N*alpha`.
For example, with 25 control-region events and transfer factor \(0.12\), the nominal signal-region background rate is \(3.0\):

```text
bin        SR
process    sig   bkg
process    0     1
rate       1.5   3.0
--------------------------
bkg_CRstat gmN 25  -  0.12[kp5]
```

This encodes the control-region counting uncertainty; keep other uncertainties on the extrapolation factor, if any, as separate nuisances (often `lnN` or a correlated parameterization). The Combine documentation specifically recommends `gmN` for a background estimated from a control region, and gives this same count-times-extrapolation construction: [Preparing the Datacard](https://cms-analysis.github.io/HiggsAnalysis-CombinedLimit/latest/part2/settinguptheanalysis/).

gmN: GPT-5.6 terra, shipped config, run 2/3

FC 0.77 rubric 5 nugget pass (supported: kp1, kp2, kp3, kp4, kp5) 3 tool calls

Use a `gmN`[kp1] (Gamma-Poisson) nuisance. It is specifically intended for the statistical uncertainty from a control-region count propagated to the signal region with an extrapolation/transfer factor.
If the control region has \(N\) observed events and the signal-region extrapolation factor is \(\alpha\), write:

```text
rate                 ...  N*alpha[kp4]   ...
bkg_CR_stat   gmN N[kp2]  ...  alpha      ...
```

Place `alpha` only in the column[kp3] for the affected signal-region background; use `-` in other process/bin columns. The nominal `rate` for that background must be exactly `N*alpha`[kp4].
For example, 4 observed control-region events with \(\alpha=0.16\) predicts \(4 \times 0.16 = 0.64\) events:

```text
rate          1.47   0.64   0.06
bkg_CR_stat   gmN 4  -      0.16   -[kp5]
```

Combine's documented counting-experiment example uses exactly this construction and recommends `gmN` for control-region or limited-MC statistical uncertainties: [Preparing the Datacard](https://cms-analysis.github.io/HiggsAnalysis-CombinedLimit/latest/part2/settinguptheanalysis/).

gmN: GPT-5.6 terra, shipped config, run 3/3

FC 0.50 rubric 5 nugget fail (supported: kp1, kp2, kp3, kp4, kp5; must_not: mn3) 5 tool calls

````
Use a **gamma-distributed (`gmN`)[kp1]** nuisance. It is specifically intended for a yield inferred from a finite
control-region count (or limited-MC count) multiplied by a transfer/extrapolation factor.
If the control region contains \(N\) observed events and the signal-region extrapolation factor is
\(\alpha\), write the nuisance row as:
```text
bkg_stat  gmN  N  -  alpha[kp5]
```
Here `N` is the integer CR count[kp2], and `alpha` is entered in the column of the background process[kp3] whose
signal-region rate is \(N\alpha\)[kp4]. For example, 25 CR events with \(\alpha=0.12\):
```text
process       sig  bkg
process       0    1
rate          3.0  3.0
bkg_stat      gmN  -    0.12[mn3]
```
The nominal background yield is \(25 \times 0.12 = 3.0\), and Combine uses the Poisson statistical
uncertainty from the 25-event control count. Do not use `lnN` for that counting uncertainty; use `lnN`
separately only if the extrapolation factor itself has an independent relative systematic uncertainty.
See the official [simple counting-experiment documentation](https://cms-analysis.github.io/HiggsAnalysis-
CombinedLimit/latest/part2/settinguptheanalysis/), which gives this control-region/extrapolation-factor
`gmN` construction.
````

gmN: GPT-5.6 terra, read-only config, run 1/3

FC 0.40 rubric 5 nugget pass (supported: kp1, kp2, kp3, kp4, kp5) 6 tool calls

Use a **`gmN`[kp1]** nuisance: it models the Poisson statistical uncertainty from the finite control-region count \(N\). If the signal-region prediction is \(N\alpha[kp4]\), write `N` after `gmN` and put the extrapolation factor \(\alpha\) in the column[kp3] of the affected background process:

```text
bin           SR
process       bkg
process       1
rate          5.0
bkg_CRstat    gmN  100   0.05[kp5]
```

Here, `100` is the observed control-region yield and `0.05` is the transfer/extrapolation factor, so the nominal SR background rate is \(100 \times 0.05 = 5.0\). Put `-` in every process column not affected by this constraint. Treat uncertainty in the extrapolation factor itself separately, typically with an `lnN` (or a shape/parametric treatment where appropriate).
This is the direct prescription in the [Combine counting-experiment documentation](https://cms-analysis.github.io/HiggsAnalysis-CombinedLimit/latest/part2/settinguptheanalysis/).

gmN: GPT-5.6 terra, read-only config, run 2/3

FC 0.45 rubric 5 nugget pass (supported: kp1, kp2, kp3, kp4, kp5, kp6) 8 tool calls

Use a **`gmN`[kp1]** nuisance. It is the recommended Gamma/Poisson constraint for the statistical uncertainty of a background predicted from a control-region event count times a transfer (extrapolation) factor. [Combine datacard documentation](https://cms-analysis.github.io/HiggsAnalysis-CombinedLimit/latest/part2/settinguptheanalysis/)
If the control region has \(N\) observed events and the signal-region extrapolation factor is \(\alpha\), write:

```text
rate                    ...  N*alpha[kp4]   ...
bkg_CR_stat     gmN N[kp2]  ...  alpha    ...
```

The `alpha` entry goes in the column[kp3] for the affected signal-region background process; use `-` in all other process/bin columns.
For example, four control-region events and \(\alpha=0.16\) predict \(4 \times 0.16 = 0.64\) signal-region events:

```text
bin             SR
process         sig     bkg
process         0       1
rate            1.50    0.64
--------------------------------
bkg_CR_stat gmN 4  -     0.16[kp5]
```

Do not enter `0.64` as the `gmN` effect: the `rate` must be \(N\alpha\), while the `gmN` row contains \(N\) after `gmN` and \(\alpha\) in the affected column. The documentation's equivalent sideband example notes the resulting relative uncertainty is \(1/\sqrt{N+1}[kp6]\) for this Gamma treatment.

gmN: GPT-5.6 terra, read-only config, run 3/3

FC 0.37 rubric 5 nugget pass (supported: kp1, kp2, kp3, kp4, kp5) 7 tool calls

````
Use a **`gmN`[kp1]** nuisance, which gives the background normalization the Poisson (gamma-constrained)
statistical uncertainty from the control-region count.
If the control region has \(N\) observed events and the transfer/extrapolation factor to the signal region
is \(\alpha\), write:
```text
bkg_stat    gmN  N[kp2]    -    alpha
```
Here `N` is the control-region event count, and `alpha` is placed under the affected background process
column. The nominal background rate for that process should be `N * alpha`[kp4].
For example, for \(N=25\), \(\alpha=0.40\), and an SR background rate of \(10\):
```text
bin           sr
process       sig     bkg
process       0       1
rate          3.2     10.0
bkg_stat      gmN 25  -       0.40[kp5]
```
Do not use `lnN` for this control-data counting uncertainty; `gmN` preserves the Poisson treatment of the
finite CR yield. Any uncertainty in the extrapolation factor itself is separate and should typically be
represented by an additional nuisance, often `lnN` (or a shape variation where applicable).
This is the convention documented in Combine's [simple counting-experiment example](https://cms-
analysis.github.io/HiggsAnalysis-CombinedLimit/latest/part2/settinguptheanalysis/#a-simple-counting-
experiment).
````

gmN: GPT-5.6 terra, no web config, run 1/3

FC 0.64 rubric 5 nugget pass (supported: kp1, kp2, kp3, kp4, kp5) 5 tool calls

````
Use a `gmN`[kp1] (gamma-distributed Poisson) nuisance. It models the finite Poisson count in the control region:
if the CR has \(N\) observed events and the signal-region transfer/extrapolation factor is \(\alpha\),
write:
```text
bkg_stat  gmN  N[kp2]  -  alpha
```
Here, `N` is the observed control-region count, `alpha` is the nominal extrapolation factor, and the
resulting nominal SR background yield is \(N\alpha[kp4]\). Put `alpha` only in the column[kp3] for the affected
background process; use `-` for all others.
For example, 25 control-region events with transfer factor 0.12:
```text
bkg_CR_stat  gmN  25  -  0.12[kp5]
```
This corresponds to an SR prediction of `25 * 0.12 = 3.0` events, with its statistical uncertainty
constrained by the control-region count. The Combine documentation explicitly gives this CR-count-times-
extrapolation use case for `gmN`: [A simple counting experiment](https://cms-
analysis.github.io/HiggsAnalysis-CombinedLimit/latest/part2/settingupthеanalysis/).
````

gmN: GPT-5.6 terra, no web config, run 2/3

FC 0.50 rubric 5 nugget pass (supported: kp1, kp2, kp3, kp4, kp5) 4 tool calls

````
Use a `gmN`[kp1] nuisance parameter. It models the Poisson statistical uncertainty from the observed control-
region count \(N\), while the entry on the affected background is the extrapolation (transfer) factor
\(\alpha\).
```text
bkg_CR_stat  gmN  25   -   0.12[kp5]
```
Here, `25` is the number of observed control-region events and `0.12` is the transfer factor for the
background in the signal-region bin/process. The nominal signal-region yield represented by this constraint
is \(25 \times 0.12 = 3.0\).
Use `-` for processes not affected by this control-region estimate. Any systematic uncertainty on the
extrapolation factor itself should be a separate nuisance, typically `lnN` (or a shape nuisance if
applicable).
This is the documented use case for `gmN`: a control-region count multiplied by an extrapolation factor into
the signal region. [Combine documentation](https://cms-analysis.github.io/HiggsAnalysis-
CombinedLimit/latest/part2/settinguptheanalysis/)
````

gmN: GPT-5.6 terra, no web config, run 3/3

FC 0.62 rubric 5 nugget pass (supported: kp1, kp2, kp3, kp4, kp5) 5 tool calls
judge wrong: mn3 missed (rate 10.0, but $25 \times 0.12 = 3.0$)

````
Use a `gmN` nuisance parameter[kp1], which models the Poisson statistical uncertainty from a finite control-
region count.
If the control region contains \(N\) observed events and the signal-region extrapolation (transfer) factor
is \(\alpha\), set the nominal signal-region background rate to \(N\alpha\)[kp4], then write:
```text
bkg_stat  gmN  N  -  alpha[kp2]
```
For example, for 25 events in the control region and \(\alpha=0.12\):
```text
rate                    10.0[mn3 judge missed]
bkg_stat  gmN  25     0.12
```
Here `gmN 25` constrains the control-region yield with its Poisson uncertainty, and `0.12` is the[kp3] scale
factor that propagates that uncertainty into the affected background rate. Put `-` in columns for processes
that the nuisance does not affect. The extrapolation-factor uncertainty itself, if any, should be
represented by a separate nuisance (often `lnN`), rather than folded into `gmN`.
This is the documented Combine prescription for a control-region count \(N\) extrapolated by \(\alpha\): [A
simple counting experiment](https://cms-analysis.github.io/HiggsAnalysis-
CombinedLimit/latest/part2/settinguptheanalysis/).
````

Shape+counting: the question and its key points

Question (agent-shape-counting-1)

I combined the two attached datacards with `combineCards.py shapes=simple-shapes-TH1.txt counting=realistic-counting-experiment.txt > combined.txt`. That worked, but running `combine -M FitDiagnostics combined.txt` on the result fails. What is the problem? (attached: simple-shapes-TH1.txt, simple-shapes-TH1.root, realistic-counting-experiment.txt)

kp1	vital	counting card has no `shapes` line, so no mapping for that channel
kp2	vital	fix: `combineCards.py -S`, or `shapes * * FAKE` in the counting card
kp3	okay	error "Shape map has no entry for channel 'counting' " at workspace building
kp4	okay	mentions both fixes
mn1	must_not	shape and counting cards cannot be combined at all
mn2	must_not	blames another cause (name clash, ROOT path, . . .)

Key points written once from the gold answer; the judge sees only these lines. Next: 24 answers to this question, in full.

Shape+counting: Qwen 3.8, shipped config, run 1/3

FC – rubric – nugget fail (no key point supported) 5 tool calls

```
(no answer: the session ended without a final message; 5 tool calls)
```

Shape+counting: Qwen 3.8, shipped config, run 2/3

FC – rubric – nugget fail (no key point supported) 4 tool calls

```
(no answer: the session ended without a final message; 4 tool calls)
```

Shape+counting: Qwen 3.8, shipped config, run 3/3

FC – rubric – nugget fail (no key point supported) 3 tool calls

```
(no answer: the session ended without a final message; 3 tool calls)
```

Shape+counting: Qwen 3.8, read-only config, run 1/3

FC 0.85 rubric 5 nugget pass (supported: kp1, kp2, kp3, kp4) 8 tool calls

Combine is on PATH, so I'll reproduce this in the shell.
The problem is that you're combining a **shape-based** datacard with a **counting** datacard, which `combineCards.py` can't do (the docs note it "will fail" -- in this version it silently produces a broken card instead).
What happened:
- `simple-shapes-TH1.txt` is shape-based (`shapes * * simple-shapes-TH1.root ...`).
- `realistic-counting-experiment.txt` is a counting datacard (no `shapes` line)[kp1].
- The combined card therefore has a `shapes` line that only covers the `shapes` channel, while the `counting` channel has no shape definitions. `combine` then dies during `text2workspace` with:

  ```
  KeyError: "Shape map has no entry for channel 'counting'"[kp3]
  ```

The fix (per the [docs, "Combination of multiple datacards"](https://cms-analysis.github.io/HiggsAnalysis-CombinedLimit/latest/part2/settinguptheanalysis/#combination-of-multiple-datacards)): add the `-S` option to[kp2] `combineCards.py`, which converts the counting card into an equivalent shape-based one by inserting a `shapes * *[kp4] FAKE` line:

```
combineCards.py -S shapes=simple-shapes-TH1.txt counting=realistic-counting-experiment.txt > combined.txt
```

I verified this works -- the combined card now has `shapes * counting FAKE` and `combine -M FitDiagnostics combined.txt` runs to completion (best fit r $\approx$ 2.5e-12, -2.55e-12/+0.349 at 68% CL).
One caveat: note the two cards share nuisance names (`lumi`, `bgnorm` appear in both), so `combineCards.py` will treat them as 100% correlated across the two channels. If that's not what you intended, rename them in one of the input cards before combining.

Shape+counting: Qwen 3.8, read-only config, run 2/3

FC 0.48 rubric 2 nugget fail (no key point supported; must_not: mn1) 0 tool calls

The problem is that you're trying to combine a **shapes** datacard and a **counting** datacard using `combineCards.py`.
These two datacard formats are fundamentally incompatible[mn1]:
- **Counting datacards** (like `realistic-counting-experiment.txt`) use simple integer counts and specify uncertainties via `lnN`, `gmN`, etc. They don't reference ROOT files.
- **Shapes datacards** (like `simple-shapes-TH1.txt`) reference ROOT files containing histograms (`$PROCESS` and `$PROCESS_$SYSTEMATIC`) and use `shape` uncertainties. They require `text2workspace.py` to build a workspace with histogram bins.
`combineCards.py` can only combine datacards of the **same type**[mn1] (all counting or all shapes). Mixing them produces a malformed datacard that `combine` cannot parse or execute.
Solution:
You need to convert both analyses to the **same format**. Typically, this means:
1. **Convert the counting experiment to a shapes datacard** by creating a ROOT file[mn1] with the expected signal/background histograms (even if they're flat/single-bin) and rewriting the counting datacard in shapes format,
OR
2. **Convert the shapes experiment to a counting datacard** if the shapes are simple enough (e.g., single-bin), though this loses the shape information.
The most common approach is to express both as **shapes datacards** with corresponding ROOT files, then combine them with `combineCards.py`.
Alternatively, if you want to keep them separate, run `combine` on each datacard independently and combine the results at the post-processing level (e.g., using `CombineHarvester` or manual combination of limit results), but this is less straightforward and not the standard workflow.
To verify: You can inspect the output of `combineCards.py` -- it will likely produce a datacard with inconsistent `shapes` and `bin`/`process` sections that `combine` rejects.

Shape+counting: Qwen 3.8, read-only config, run 3/3

FC – rubric – nugget fail (no key point supported) 65 tool calls
loop: same command 45 times, timeout at 400 s, no answer

```
(no answer: timeout; 65 tool calls)
```

Shape+counting: GPT-5.6 luna, shipped config, run 1/3

FC 0.07 rubric 1 nugget fail (no key point supported) 6 tool calls

killed: bash permission auto-reject; only narration left

```
I'll reproduce the failure from the supplied cards, then inspect the combined-card parsing error to
distinguish a datacard inconsistency from a missing shape-file or workspace issue.The likely failure is in
the merged card's nuisance/shape bookkeeping, so I'm reproducing the merge and `FitDiagnostics` locally
against the supplied ROOT file to identify the exact error rather than infer it from the card text.
```

Shape+counting: GPT-5.6 luna, shipped config, run 2/3, page 1/2

FC 0.70 rubric 5 nugget pass (supported: kp1, kp2, kp3, kp4) 16 tool calls

I'll reproduce the combination and `FitDiagnostics` run, then inspect the exact failure and the combined card for cross-channel naming or shape-file issues.The likely issue is a structural collision introduced by combining the cards, not the statistical model itself. I'm checking the generated card and reproducing `FitDiagnostics` against the real shape file so the exact parser error is clear.The Combine executable is available locally, so I'm reproducing the card combination and fit in the supplied environment. The key checks are the combined channel/process names and whether the shape card's ROOT references survive the combination.The combination itself is structurally valid, but the two cards introduce a subtle naming collision: both use the channel name `bin1`. I'm reproducing the merged card with the local Combine installation to distinguish that from any ROOT-shape or workspace error.The counting card has no `shapes` [kp1] mapping. `combineCards.py` can concatenate it with the shape card, but the resulting card still has shape information only for the `shapes` channel. During `FitDiagnostics`, Combine tries to build shapes for the `counting` channel and fails with:

```text
KeyError: "Shape map has no entry for channel 'counting'" [kp3]
```

Convert the counting card to an equivalent shape card when combining by adding `-S` [kp2]:

```bash
combineCards.py -S \
  shapes=simple-shapes-TH1.txt \
  counting=realistic-counting-experiment.txt > combined.txt
combine -M FitDiagnostics combined.txt
```

The `-S` [kp2] option adds:

FC 0.70 rubric 5 nugget pass (supported: kp1, kp2, kp3, kp4) 16 tool calls

````
```text
shapes * counting FAKE
```
to the combined card. This tells Combine to use fake one-bin shapes for the counting channel while retaining
the real ROOT shapes for the `shapes` channel.
This behavior is documented in [Combination of multiple datacards](https://cms-
analysis.github.io/HiggsAnalysis-CombinedLimit/latest/part2/settinguptheanalysis/#combination-of-multiple-
datacards), which specifically states that counting cards must be converted with `shapes * * FAKE[kp4]` or with
`combineCards.py -S` before combining them with shape cards.
````

Shape+counting: GPT-5.6 luna, shipped config, run 3/3, page 1/2

FC 0.67 rubric 5 nugget pass (supported: kp1, kp2, kp3) 15 tool calls

I'm checking the generated-card mechanics and the exact Combine validation likely triggered by mixing a shape-based channel with the counting channel. The key distinction is whether the failure is from the combined datacard content or from the ROOT shape file path/histograms.The failure is likely at the combination boundary rather than in `FitDiagnostics`: Combine treats shape and counting cards differently, and the combined card must preserve valid channel-specific shape references. I'm checking the generated card and reproducing the exact command to identify the concrete error.The combined card is structurally suspicious: its single `shapes *` rule is inherited from the shape card and is also applied to the counting channel. I'm reproducing the failure locally, then I'll verify the documented combination fix.The counting card was not converted to a shape card. The combined datacard contains a `shapes` rule only for the `shapes` channel:

```text
shapes * shapes attachments/simple-shapes-TH1.root $PROCESS $PROCESS_$SYSTEMATIC
```

There is no shape rule for the `counting` channel. Consequently, `combine` fails with:

```text
KeyError: "Shape map has no entry for channel 'counting'[kp3]"
```

Convert the counting card to a fake one-bin shape card when combining:

```bash
combineCards.py -S[kp2] \
  shapes=attachments/simple-shapes-TH1.txt \
  counting=attachments/realistic-counting-experiment.txt \
  > combined-shape.txt
```

Shape+counting: GPT-5.6 luna, shipped config, run 3/3, page 2/2

FC 0.67 rubric 5 nugget pass (supported: kp1, kp2, kp3) 15 tool calls

````
```
The resulting card should contain an additional line like:
```text
shapes * counting FAKE
```
Then run:
```bash
combine -M FitDiagnostics combined-shape.txt
```
The `-S` option adds the `FAKE` shapes declaration for counting channels. This is required because a
combined workspace must provide a shape mapping for every channel, even when one channel originated as a
counting experiment. The official documentation describes this conversion and the equivalent manual fix:
[Combination of multiple datacards](https://cms-analysis.github.io/HiggsAnalysis-
CombinedLimit/latest/part2/settinguptheanalysis/#combination-of-multiple-datacards).
````
````

Shape+counting: GPT-5.6 luna, read-only config, run 1/3

FC 0.07 rubric 1 nugget fail (no key point supported) 7 tool calls

killed: external_directory reject after a path typo; only narration left

```
I'll reproduce the combined-card workflow and inspect the generated card's shape and nuisance entries. The
key distinction is whether the failure is from the ROOT shape file/path or from an invalid merged nuisance-
parameter definition.I'll inspect the generated combined card and the recorded Combine output, then
reproduce the fit locally against the provided ROOT shapes so the diagnosis is tied to the actual failure
rather than just the card syntax.
```

Shape+counting: GPT-5.6 luna, read-only config, run 2/3, page 1/2

FC 0.87 rubric 5 nugget fail (supported: kp1, kp3, kp4) 14 tool calls

judge wrong: kp2 missed, the -S/FAKE fix is there

I'm checking the combined-card mechanics and the shape-file references, since `combineCards.py` can succeed even when the resulting card is not self-contained for `combine`.The likely issue is in how `combineCards.py` namespaces channels and shape references, but the exact failure depends on the generated card and ROOT contents. I'm checking the available Combine executable and the combined-card structure before pinning down the fix.The combined card reveals the issue: it mixes a shape-based channel with a counting-only channel, but the generated card has only the shape file declaration. I'm confirming the exact parser failure and the documented correction, including whether `-S` or an explicit `FAKE` shape line is the cleanest fix.The problem is that `combined.txt` mixes a shape-based channel (`shapes`) with a counting-only[kp1] channel (`counting`):

```text
shapes *       shapes  simple-shapes-TH1.root ...
```

This declares shapes only for the `shapes` channel. However, after combining, Combine treats the entire card as shape-based because one input card uses shapes. It therefore tries to load shapes for the `counting` channel too, producing:

```text
KeyError: "Shape map has no entry for channel 'counting'"
```[kp3]

Convert the counting card into an equivalent shape card by adding a `FAKE` shapes directive, or regenerate[kp2 judge missed] the combination with `-S`:

```bash
combineCards.py -S \
  shapes=simple-shapes-TH1.txt \
```

FC 0.87 rubric 5 nugget fail (supported: kp1, kp3, kp4) 14 tool calls
judge wrong: kp2 missed, the -S/FAKE fix is there

````
  counting=realistic-counting-experiment.txt > combined.txt
```
The resulting card should contain both directives:
```text
shapes * counting FAKE
shapes * shapes simple-shapes-TH1.root $PROCESS $PROCESS_$SYSTEMATIC
```
Alternatively, add this line to the counting card[kp4] immediately after its `imax`, `jmax`, and `kmax` header:
```text
shapes * * FAKE
```
Then combine without `-S`. The `FAKE` mapping supplies one-bin shapes for the counting channel, while the
original ROOT mapping remains in use for the shape channel.
This is the documented requirement for combining a shape datacard with a counting datacard: [Combination of
multiple datacards](https://cms-analysis.github.io/HiggsAnalysis-
CombinedLimit/latest/part2/settinguptheanalysis/#combination-of-multiple-datacards). I also reproduced the
failure with `combined.txt`; the version with the `FAKE` directive runs past datacard conversion
successfully.
````
````

Shape+counting: GPT-5.6 luna, read-only config, run 3/3, page 1/2

FC 0.86 rubric 5 nugget pass (supported: kp1, kp2, kp3, kp4) 14 tool calls

I'll reproduce the combination and fit, then inspect the combined card and the ROOT shape references to identify whether the failure is naming, workspace construction, or an actual fit issue.I'm checking the generated combined card and reproducing the failure so the diagnosis is based on the actual parser error, not just the card syntax.The local Combine environment is available, so I'll reproduce the failure against the generated card and inspect the exact parser/fitting error rather than infer it from the card structure alone.The failure is a shape-map issue, not a statistical incompatibility between the cards. The combined card has a shape-based `shapes` mapping for `shapes`, but no mapping for the counting channel, so `FitDiagnostics` cannot construct a workspace for `counting`; I'm checking the minimal `FAKE` mapping fix now.The problem is that you combined a **shape-based** datacard with a **counting** datacard.
`combineCards.py` produced a card containing only the shape mapping

```text
shapes * shapes simple-shapes-TH1.root $PROCESS $PROCESS_$SYSTEMATIC
```

but the `counting` channel has no shape mapping. `FitDiagnostics` therefore tries to construct shapes for channel `counting` and fails with:

```text
KeyError: "Shape map has no entry for channel 'counting'[kp3]"
```

Convert the counting card to an equivalent one-bin shape card when combining:

```bash
combineCards.py -S[kp2] \
  shapes=simple-shapes-TH1.txt \
  counting=realistic-counting-experiment.txt > combined.txt
```

Shape+counting: GPT-5.6 luna, read-only config, run 3/3, page 2/2

FC 0.86 rubric 5 nugget pass (supported: kp1, kp2, kp3, kp4) 14 tool calls

````
```
Alternatively, add this line to `realistic-counting-experiment.txt`, after the `imax`, `jmax`, and `kmax`
lines:
```text
shapes * * FAKE[kp4]
```
Then combine without `-S`:
```bash
combineCards.py \
  shapes=simple-shapes-TH1.txt \
  counting=realistic-counting-experiment.txt > combined.txt
```
`FAKE` does not introduce fake data. It tells Combine to represent the counting channel as a one-bin shape
model. I confirmed that adding the equivalent mapping allows `combine -M FitDiagnostics` to run
successfully.
This is documented in [Combination of multiple datacards](https://cms-analysis.github.io/HiggsAnalysis-
CombinedLimit/latest/part2/settingup theanalysis/) and [Normalizations](https://cms-
analysis.github.io/HiggsAnalysis-CombinedLimit/latest/part3/nonstandard/#normalizations).
````
````

Shape+counting: GPT-5.6 luna, no web config, run 1/3, page 1/2

FC 0.79 rubric 5 nugget pass (supported: kp1, kp2, kp3, kp4) 16 tool calls

The likely failure is in the interaction between the shape-based card and the counting card, so I'm checking the combined-card conventions and reproducing the exact Combine error before identifying the fix.The failure is likely in the combined card's nuisance-parameter naming, so I'm checking the generated card and reproducing the exact `FitDiagnostics` failure before narrowing the fix.The failure is likely in the generated combined card rather than `FitDiagnostics` itself. I'm checking the exact combined-card paths and reproducing from the attachment directory so the referenced ROOT shape file resolves correctly.The counting card has no `shapes` [kp1] declaration. `combineCards.py` combined the text successfully, but the resulting `combined.txt` contains only:

```text
shapes * shapes simple-shapes-TH1.root $PROCESS $PROCESS_$SYSTEMATIC
```

That mapping applies to the `shapes` channel only. When `FitDiagnostics` processes the `counting` channel, it cannot find a shape map and reports:

```text
Shape map has no entry for channel 'counting' [kp3]
```

Use `-S` [kp2] when combining, which adds the required `FAKE` shape mapping for counting channels:

```bash
combineCards.py -S \
  shapes=simple-shapes-TH1.txt \
  counting=realistic-counting-experiment.txt > combined.txt
combine -M FitDiagnostics combined.txt
```

FC 0.79 rubric 5 nugget pass (supported: kp1, kp2, kp3, kp4) 16 tool calls

````
The corrected combined card should contain:
```text
shapes *          counting  FAKE
shapes *          shapes    simple-shapes-TH1.root $PROCESS $PROCESS_$SYSTEMATIC
```
Alternatively, add this line after the `imax`, `jmax`, and `kmax` header in `realistic-counting-
experiment.txt`:
```text
shapes * * FAKE[kp4]
```
The `FAKE` mapping converts the counting card into an equivalent one-bin shape card. This is the documented
procedure for combining shape and counting datacards: [Combination of multiple datacards](https://cms-
analysis.github.io/HiggsAnalysis-CombinedLimit/latest/part2/settinguptheanalysis/#combination-of-multiple-
datacards).
````

Shape+counting: GPT-5.6 luna, no web config, run 2/3, page 1/2

FC 0.89 rubric 5 nugget pass (supported: kp1, kp2, kp3, kp4) 14 tool calls

I'll inspect the combination semantics and reproduce the failure path, focusing on how `combineCards.py` namespaces channels and shape references across the two cards.I'm checking the combined-card mechanics and reproducing the failure so the diagnosis distinguishes a datacard incompatibility from a missing/invalid shape input.The likely issue is in how `combineCards.py` rewrites the shape-based card, but I'll verify the generated card and the actual Combine error before narrowing it down.The combined card shows the key distinction: it contains the shape-file directive for the `shapes` channel, but no `FAKE` directive for the counting channel. I'll reproduce the failure locally, then compare it with the documented `-S` conversion path.The generated card is syntactically produced, but it lacks a shape mapping for the counting channel. I reproduced the failure: Combine builds the shape model for `counting`, then finds no entry because the original `combineCards.py` invocation did not request conversion of the counting card to a fake shape card. I'm verifying the corrected combined card against the actual ROOT input now.The combined datacard contains two different channel types:

- `shapes` is a shape-based channel and has a `shapes` mapping to `simple-shapes-TH1.root`.
- `counting` is a counting-only channel and has no `shapes` mapping[kp1].

`combineCards.py` allows the combination, but the resulting card is not valid for `text2workspace.py`/`FitDiagnostics`. Combine tries to load shapes for the `counting` channel and fails with:

```text
KeyError: "Shape map has no entry for channel 'counting'"[kp3]
```

Convert the counting card to an equivalent shape card by adding:

```text
shapes * * FAKE[kp2]
```

FC 0.89 rubric 5 nugget pass (supported: kp1, kp2, kp3, kp4) 14 tool calls

````
```
to `realistic-counting-experiment.txt`, or use `combineCards.py -S`[kp4]:
```bash
combineCards.py -S \
  shapes=attachments/simple-shapes-TH1.txt \
  counting=attachments/realistic-counting-experiment.txt \
  > combined.txt
combine -M FitDiagnostics combined.txt
```
The `-S` option inserts the required fake-shape mapping for counting channels. This is the documented
solution: [Combination of multiple datacards](https://cms-analysis.github.io/HiggsAnalysis-
CombinedLimit/latest/part2/settinguptheanalysis/#combination-of-multiple-datacards).
The existing `combined-shapes.txt` demonstrates the corrected structure:
```text
shapes *         counting  FAKE
shapes *         shapes    simple-shapes-TH1.root $PROCESS $PROCESS_$SYSTEMATIC
```
I reproduced the failure with the original combined card and confirmed that the `-S` version runs through
`FitDiagnostics`.
````
````

Shape+counting: GPT-5.6 luna, no web config, run 3/3, page 1/2

FC 0.85 rubric 5 nugget pass (supported: kp1, kp2, kp3, kp4) 16 tool calls

I'll reproduce the combined-card failure, then inspect the resulting datacard structure and Combine's validation error to identify whether the issue is in the card syntax or the ROOT shape inputs.I'm checking the combined-card syntax and the referenced shape file, then I'll reproduce the failure if the local Combine environment is available.The Combine executable is available locally, so I'll reproduce the failure against the supplied combined card and inspect the merged shape declarations. The key distinction is whether the failure is in the card merge or in the shape-based likelihood construction.The combined card shows the likely issue: its wildcard `shapes` mapping applies only to the `shapes` channel, while the counting channel remains a counting channel. I'm reproducing the exact failure locally and checking the generated ROOT inputs to distinguish a missing histogram from a card-format problem.The generated card mixes a shape channel with a counting channel, but it contains a shape mapping only for `shapes`. I'm checking the documented conversion path and the corrected card behavior before stating the fix.The problem is that the two cards use different likelihood types:

- `simple-shapes-TH1.txt` is a shape-based card because it contains a `shapes` mapping.
- `realistic-counting-experiment.txt` is a counting card and has no shape mapping.

The combined card therefore contains only:

```text
shapes * shapes simple-shapes-TH1.root $PROCESS $PROCESS_$SYSTEMATIC
```

This maps shapes for the `shapes` channel, but not for the `counting` channel. During conversion to a workspace, Combine tries to treat the whole combination as shape-based and reports:

```text
KeyError: "Shape map has no entry for channel 'counting'" kp3
```

FC 0.85 rubric 5 nugget pass (supported: kp1, kp2, kp3, kp4) 16 tool calls

````
Convert the counting card to an equivalent one-bin shape card while combining:
```bash
combineCards.py -S[kp2] \
  shapes=simple-shapes-TH1.txt \
  counting=realistic-counting-experiment.txt > combined.txt
```
This adds the required mapping:
```text
shapes * counting FAKE
```
Alternatively, add that line manually after the `kmax` section in the combined card. The `FAKE` mapping
makes the counting channel a one-bin shape analysis, allowing it to coexist with the histogram-based
channel.
This is the documented behavior for combining shape and counting cards: [`combineCards.py` combination
documentation](https://cms-analysis.github.io/HiggsAnalysis-
CombinedLimit/latest/part2/settinguptheanalysis/#combination-of-multiple-datacards). The `FAKE` conversion
is also described under [normalizations](https://cms-analysis.github.io/HiggsAnalysis-
CombinedLimit/latest/part3/nonstandard/#normalizations).
````

Shape+counting: GPT-5.6 terra, shipped config, run 1/3, page 1/2

FC 0.82 rubric 5 nugget pass (supported: kp1, kp2, kp3, kp4) 15 tool calls

I'll reproduce the combined-card fit against the attached ROOT shapes, then use the resulting error to identify the invalid combined model definition.I'll reproduce the fit against the existing combined card, then trace the failing object or nuisance definition back to the merged inputs.The failure is reproduced during workspace construction, before the fit: the merged card supplies a shape mapping only for `shapes`, while the `counting` channel has none. I'm checking the supported way to explicitly mark that channel as count-only and will validate the corrected card locally.The failure is reproduced during workspace construction, before any fit: the combined card contains a real `shapes` directive only for the `shapes` channel, but the `counting` channel has no shape-map entry. I'm checking the documented mixed-card convention to give the exact minimal correction.The failure is reproduced: the combined card has a `shapes` mapping only for `shapes`, while FitDiagnostics builds shapes for every channel and cannot resolve `counting`. I'll regenerate it with `combineCards.py -S`[kp2], which explicitly converts the counting channel to a fake shape channel, then confirm the fit starts.The combined card mixes a shape-based channel (`shapes`) with a pure counting channel (`counting`). `combineCards.py` can write a syntactically valid combined card without converting the counting channel, but `FitDiagnostics` then treats the model as shape-based and requires a shape-map entry for every channel. `counting` has none, causing:

```text
KeyError: "Shape map has no entry for channel 'counting'"[kp3]
```

Convert the counting card to a fake shape channel during combination:

```bash
combineCards.py -S[kp2] shapes=simple-shapes-TH1.txt counting=realistic-counting-experiment.txt > combined.txt
combine -M FitDiagnostics combined.txt
```

Shape+counting: GPT-5.6 terra, shipped config, run 1/3, page 2/2

FC 0.82 rubric 5 nugget pass (supported: kp1, kp2, kp3, kp4) 15 tool calls

````
I verified this command succeeds locally and obtains a fit result.
Equivalently, add this after `imax`/`jmax`/`kmax` in the counting datacard before combining:
```text
shapes * * FAKE[kp4]
```
This is the documented procedure for combining shape and counting datacards: [Combination of multiple
datacards](https://cms-analysis.github.io/HiggsAnalysis-CombinedLimit/latest/part2/settinguptheanalysis/).
````

Shape+counting: GPT-5.6 terra, shipped config, run 2/3

FC 0.88 rubric 5 nugget pass (supported: kp1, kp2, kp3, kp4) 11 tool calls

I'll reproduce the merged-card fit against the supplied ROOT shapes and inspect the exact failure before attributing it to the card syntax.I'll reproduce the fit against the attached cards locally and inspect the exact Combine diagnostic, then trace it to the combined-card structure.I'll reproduce the fit using the existing merged card and inspect the actual Combine error, then trace it to the card construction.`FitDiagnostics` fails while converting the combined card: the shape declaration applies only to the renamed `shapes` channel, but the `counting` channel has none. I'm checking the documented way to mark the counting channel as rate-only and will verify the corrected combined card.`combineCards.py` produced text, but it combined incompatible card types: `shapes` is a shape-based card, while `counting` is a pure counting card. The resulting `combined.txt` has a `shapes` directive only for the `shapes` channel, so when `FitDiagnostics` builds the shape model it cannot find a shape mapping for channel `counting`:

```text
KeyError: "Shape map has no entry for channel 'counting'[kp3]"
```

Convert the counting card to a fake shape card during combination:

```bash
combineCards.py -S[kp2] shapes=simple-shapes-TH1.txt \
  counting=realistic-counting-experiment.txt > combined.txt
combine -M FitDiagnostics combined.txt
```

Equivalently, add this line immediately after the `imax`/`jmax`/`kmax` section of `realistic-counting-experiment.txt` before combining:

```text
shapes * * FAKE[kp4]
```

`FAKE` does not require a ROOT histogram file; it represents the counting channel as an equivalent one-bin shape model. This is the documented mechanism for combining a counting datacard with a shape datacard, and is specifically supported by `combineCards.py -S[kp2]`: [Combine documentation](https://cms-analysis.github.io/HiggsAnalysis-CombinedLimit/latest/part2/settinguptheanalysis/).

Shape+counting: GPT-5.6 terra, shipped config, run 3/3

FC 0.85 rubric 5 nugget pass (supported: kp1, kp2, kp3, kp4) 7 tool calls

I'll reproduce the fit against the generated-style inputs and inspect the exact Combine diagnostic, then distinguish any datacard issue from a relative-path issue for the shape ROOT file.I'll reproduce the combined-card workflow against the attached shape file, then inspect the exact FitDiagnostics failure and validate it against the documented combination behavior.The combined card mixes a histogram-based channel with a pure counting channel. `combineCards.py` has emitted the latter without a `shapes`[kp1] mapping, which causes `text2workspace.py` to fail while building the combined model. I'm confirming the supported conversion flag and its resulting card.The combined card mixes a shape-based channel with a counting channel. Although `combineCards.py` writes a combined card, it does **not** automatically give the counting channel a `shapes` entry. Consequently, while building the workspace, Combine treats the card as shape-based but cannot find shapes for the `counting` channel:

```text
KeyError: "Shape map has no entry for channel 'counting'[kp3]"
```

Create the combined card with `-S`[kp2], which inserts the required `shapes * * FAKE`[kp4] mapping for the counting card:

```bash
combineCards.py -S shapes=simple-shapes-TH1.txt counting=realistic-counting-experiment.txt > combined.txt
combine -M FitDiagnostics combined.txt
```

I verified this corrected command completes successfully. The official documentation explicitly notes that combining shape and counting cards requires converting the counting card to an equivalent shape-based form, either by adding `shapes * * FAKE`[kp4] manually or using `combineCards.py -S`: [Combination of multiple datacards](https://cms-analysis.github.io/HiggsAnalysis-CombinedLimit/latest/part2/settingup theanalysis/#combination-of-multiple-datacards).

Shape+counting: GPT-5.6 terra, read-only config, run 1/3, page 1/2

FC 0.64 rubric 5 nugget pass (supported: kp1, kp2, kp3) 12 tool calls

I'll reproduce the merged-card fit against the attached shape file, then trace the exact parsing or model error to the datacard definition.I'll reproduce the fit using the supplied cards in the local Combine environment, then inspect the exact failure against the datacard's shape and nuisance definitions.`combined.txt` mixes a template-shape channel with a counting channel, yet it contains no `FAKE` shape mapping for `counting`. I'm reproducing the fit against the generated card to capture the precise failure.The merged card retains a ROOT-shape mapping only for `shapes`; its `counting` channel has no shape map. I reproduced the reported failure: workspace creation stops with `KeyError: "Shape map has no entry for channel 'counting'"`.The cards were combined as a shape-based model, but `combined.txt` has no `shapes`[kp1] mapping for the `counting` channel. As a result, `text2workspace.py` tries to load a shape for that channel and fails with:

```text
KeyError: "Shape map has no entry for channel 'counting'[kp3]"
```

`combineCards.py` does not automatically make a counting card compatible with a shape card unless you request it. Recreate the combined card with `-S`[kp2]:

```bash
combineCards.py -S shapes=simple-shapes-TH1.txt counting=realistic-counting-experiment.txt > combined-shapes.txt
combine -M FitDiagnostics combined-shapes.txt
```

`-S`[kp2] adds:

```text
shapes * counting FAKE
```

Shape+counting: GPT-5.6 terra, read-only config, run 1/3, page 2/2

FC 0.64 rubric 5 nugget pass (supported: kp1, kp2, kp3) 12 tool calls

```
which represents the counting channel as a one-bin shape channel. I confirmed that `FitDiagnostics` runs
successfully with this corrected combined card.
The shared nuisance `lumi` remains correlated across the two channels because it has the same name and PDF
type in both input cards; all differently named nuisances remain uncorrelated. Combine's documentation
explicitly describes the `-S`kp2 conversion requirement for mixing shape and counting cards: [Combination of
multiple datacards](https://cms-analysis.github.io/HiggsAnalysis-
CombinedLimit/latest/part2/settinguptheanalysis/#combination-of-multiple-datacards).
```

Shape+counting: GPT-5.6 terra, read-only config, run 2/3, page 1/2

FC 0.92 rubric 5 nugget pass (supported: kp1, kp2, kp3, kp4) 16 tool calls

I'll reproduce the fit against the attached cards and inspect the generated combined card, then verify the failing requirement against the Combine documentation/source.I'm reproducing the fit against the generated combined card and inspecting the merged nuisance definitions; the shape card's `sigma` convention is a likely constraint conflict.The failure occurs during workspace construction, before the fit: the combined card contains a shape mapping only for the `shapes` channel, but also asks Combine to model the `counting` channel. I'm confirming the minimal explicit `FAKE` shape mapping for the counting-only channel.The combined datacard mixes a template-shape channel (`shapes`) with a counting channel (`counting`), but `combined.txt` only has a `shapes` mapping for the former:

```text
shapes * shapes simple-shapes-TH1.root $PROCESS $PROCESS_$SYSTEMATIC
```

`FitDiagnostics` builds a shape-based model because one channel uses templates, then cannot find a shape mapping for the `counting` channel:

```text
KeyError: "Shape map has no entry for channel 'counting'[kp3]"
```

Convert the counting card to an equivalent one-bin shape channel during combination:

```bash
combineCards.py -S[kp2] shapes=simple-shapes-TH1.txt counting=realistic-counting-experiment.txt > combined.txt
combine -M FitDiagnostics combined.txt
```

Equivalently, add this immediately after the `imax`/`jmax`/`kmax` block of `realistic-counting-experiment.txt` before combining:

FC 0.92 rubric 5 nugget pass (supported: kp1, kp2, kp3, kp4) 16 tool calls

````
```text
shapes * * FAKE[kp4]
```
This does not require a ROOT file or change the counting likelihood; it supplies Combine's required fake
shape mapping. The expected output has both mappings, as in your `combined-shapes.txt`:
```text
shapes * counting FAKE
shapes * shapes simple-shapes-TH1.root $PROCESS $PROCESS_$SYSTEMATIC
```
This is explicitly required when combining a shape and counting datacard in the [Combine
documentation](https://cms-analysis.github.io/HiggsAnalysis-
CombinedLimit/latest/part2/settingup theanalysis/#combination-of-multiple-datacards).
````

Shape+counting: GPT-5.6 terra, read-only config, run 3/3, page 1/2

FC 0.83 rubric 5 nugget pass (supported: kp1, kp2, kp3, kp4) 12 tool calls

I'll reproduce the combined-card fit against the supplied ROOT shapes and inspect the exact Combine diagnostic before identifying the card-level issue.The shape card makes the combined model a binned likelihood, while the counting card has no shape declaration. I'll reproduce it locally against the attached files to capture the exact failure, then verify the documented remedy.The shape card activates template-based inputs, while the counting card has no shape mapping. I'm reproducing the combined fit against the supplied artifacts to identify the precise parser/model-building failure and verify the correction.The combined card mixes a shape-based channel (`shapes`) with a counting channel (`counting`) but provides a `shapes` directive only for the former. Consequently `text2workspace.py`, invoked by `FitDiagnostics`, tries to obtain a template for `counting` and fails with:

```text
KeyError: "Shape map has no entry for channel 'counting'" [kp3]
```

`combineCards.py` can write this syntactically, but the result is not a usable mixed shape/counting model unless the counting card is converted to a fake shape channel. Use `-S` [kp2] when combining:

```bash
combineCards.py -S shapes=simple-shapes-TH1.txt \
  counting=realistic-counting-experiment.txt > combined.txt
combine -M FitDiagnostics combined.txt
```

Equivalently, add this line to the counting card before combining:

```text
shapes * * FAKE [kp4]
```

FC 0.83 rubric 5 nugget pass (supported: kp1, kp2, kp3, kp4) 12 tool calls

````
I confirmed locally that the original `attachments/combined.txt` fails with the quoted `KeyError`, while the
provided `combined_S.txt`, which contains:
```text
shapes * counting FAKE
```
runs successfully with `FitDiagnostics`.
The documented requirement and `-S` kp2 solution are in the [Combine datacard-combination
documentation](https://cms-analysis.github.io/HiggsAnalysis-
CombinedLimit/latest/part2/settinguptheanalysis/).
````

Shape+counting: GPT-5.6 terra, no web config, run 1/3, page 1/2

FC 0.85 rubric 5 nugget pass (supported: kp1, kp2, kp3) 13 tool calls

I'll reproduce the combined card and inspect the exact `FitDiagnostics` failure, including the referenced shape file.I'll reproduce the combined card and `FitDiagnostics` locally, then trace the reported failure to the conflicting datacard definition.I'll reproduce the fit against the generated combined card and inspect the exact parser or model-construction failure, then verify the relevant shape-systematic rule in the Combine documentation.`FitDiagnostics` fails while building the combined model, before any fit: the merged card makes the counting channel inherit a histogram shape requirement. I'm checking the generated card against the shape-card syntax to identify the minimal correction.`combineCards.py` propagated the shape-card wildcard rule into the combined card. It therefore tries to load histograms for the counting channel, which has none; I'm confirming the supported fake-shape remedy.The merged card applies the shape-card wildcard `shapes * *` only to the `shapes` channel. The counting channel has no shape mapping, which is why model construction stops before fitting; I'm confirming the minimal fake-shape mapping correction.The combined card applies the shape declaration only to the `shapes` channel:

```text
shapes * shapes simple-shapes-TH1.root $PROCESS $PROCESS_$SYSTEMATIC
```

but the `counting` channel has no shape mapping. Once any `shapes` directive exists, Combine builds the model through the shape machinery and attempts to retrieve a shape for every channel. It therefore fails with:

```text
KeyError: "Shape map has no entry for channel 'counting'" kp3
```

Declare the counting channel as a fake shape channel. Generate the combination with:

```bash
```

FC 0.85 rubric 5 nugget pass (supported: kp1, kp2, kp3) 13 tool calls

````
combineCards.py -S[kp2] shapes=simple-shapes-TH1.txt counting=realistic-counting-experiment.txt > combined.txt
combine -M FitDiagnostics combined.txt
```
`-S` adds the required:
```text
shapes * counting FAKE
```
before the real mapping for the histogram channel. I verified this equivalent card (`combined-shapes.txt`)
runs `FitDiagnostics` successfully.
The ROOT `TNetXNGFile` line is unrelated noise from ROOT; the fatal problem is the missing `counting` shape
map. This is the standard solution also recommended in the Combine discussion: [cms-talk topic
19916](https://cms-talk.web.cern.ch/t/19916). The documented `combineCards.py` syntax is at [Combination of
multiple datacards](https://cms-analysis.github.io/HiggsAnalysis-
CombinedLimit/latest/part2/settinguptheanalysis/#combination-of-multiple-datacards).
````
````

Shape+counting: GPT-5.6 terra, no web config, run 2/3, page 1/2

FC 0.86 rubric 5 nugget pass (supported: kp1, kp2, kp3, kp4) 9 tool calls

I'll reproduce the combined-card fit against the supplied files, then inspect the exact failure and validate it against Combine's documented datacard rules.I'll reproduce the merged card and the fit locally, then inspect the generated nuisance definitions and the exact FitDiagnostics error.`combineCards.py` produces a syntactically valid mixed card. I'm reproducing the fit against the supplied inputs to capture the precise failure; the likely issue is an unconstrained shape nuisance exposed by FitDiagnostics.I'm reproducing this against the generated card. The combination itself preserves a shape input for `shapes` but leaves `counting` as a counting channel; the FitDiagnostics failure will identify which model conversion it requires.The combination retains a global `shapes` directive for the `shapes` channel only. `FitDiagnostics` then tries to build templates for every channel and has no shape mapping for `counting`; I am checking the documented conversion workflow and will validate the corrected card.`FitDiagnostics` fails while converting the combined card because its `counting` channel has no `shapes`[kp1] mapping. I reproduced the `KeyError: "Shape map has no entry for channel 'counting'"`; the `TNetXNGFile` line is incidental to that failed shape-model construction.The combined card mixes a shape-based channel (`shapes`) with a counting-only channel (`counting`) but contains no `shapes` mapping for `counting`. Since the presence of any `shapes` directive makes Combine build a shape model, `FitDiagnostics` looks up shapes for every channel and fails with:

```text
KeyError: "Shape map has no entry for channel 'counting'[kp3]"
```

`combineCards.py` should be invoked with `-S`[kp2], which creates the required fake shape mapping for counting channels:

```bash
combineCards.py -S shapes=simple-shapes-TH1.txt counting=realistic-counting-experiment.txt > combined.txt
combine -M FitDiagnostics combined.txt
```

FC 0.86 rubric 5 nugget pass (supported: kp1, kp2, kp3, kp4) 9 tool calls

````
```
The resulting card includes:
```text
shapes * counting FAKE
```
Alternatively, add `shapes * * FAKE`[kp4] to the counting datacard before combining. The Combine documentation
explicitly prescribes `-S`[kp2] (or the `FAKE` directive) for shape-plus-counting combinations: [Combination of
multiple datacards](https://cms-analysis.github.io/HiggsAnalysis-
CombinedLimit/latest/part2/settinguptheanalysis/#combination-of-multiple-datacards).
````
````

Shape+counting: GPT-5.6 terra, no web config, run 3/3, page 1/2

FC 0.80 rubric 5 nugget fail (supported: kp1, kp3, kp4) 13 tool calls

judge wrong: kp2 missed, the -S fix is there

I'll reproduce the combined card's validation failure and trace it to the relevant datacard semantics, including the shape-file inputs.I'll reproduce the combined-card fit against the supplied files and inspect the resulting diagnostic before identifying the card-level incompatibility.I'll reproduce the fit against the generated card and inspect the merged `shapes` directives and exact Combine error.The merged card preserves the shape directive only for the `shapes` channel, but workspace conversion is attempting to obtain a shape for `counting`. I'm checking the generated alternative card and Combine's shape-card rules to distinguish the merge issue from the input's shape configuration.`combineCards.py` produced a syntactically valid card, but it did **not** make the counting channel shape-compatible.

Your combined card contains only:

```text
shapes * shapes simple-shapes-TH1.root $PROCESS $PROCESS_$SYSTEMATIC
```

This supplies templates for the `shapes` channel only.[kp1] When `FitDiagnostics` builds the model, it also tries to resolve a shape mapping for the `counting` channel and fails with:

```text
KeyError: "Shape map has no entry for channel 'counting'"[kp3]
```

Create the combination with `-S`[kp2 judge missed], which inserts a fake-shape mapping for counting channels:

```bash
combineCards.py -S shapes=simple-shapes-TH1.txt counting=realistic-counting-experiment.txt > combined.txt
combine -M FitDiagnostics combined.txt
```

The resulting card will include:

Shape+counting: GPT-5.6 terra, no web config, run 3/3, page 2/2

FC 0.80 rubric 5 nugget fail (supported: kp1, kp3, kp4) 13 tool calls

judge wrong: kp2 missed, the -S fix is there

````
```text
shapes * counting FAKE
```
I confirmed that `FitDiagnostics` runs successfully with that mapping. The Combine documentation explicitly
notes that shape and counting cards require this conversion, either by adding `shapes * * FAKE` to the
counting card or by using `combineCards.py -S`[kp4]: [Combination of multiple datacards](https://cms-
analysis.github.io/HiggsAnalysis-CombinedLimit/latest/part2/settinguptheanalysis/).
````